

Reviewer Pack — Matroid Rank and Karchmer-Wigderson Communication Complexity

Entry 028167524e77 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 05:07:51 UTC

Matroid Rank and Karchmer-Wigderson Communication Complexity

Entry ID: 028167524e77 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 05:07:51 UTC

1. Conjecture

Field A (mathematical branch): Matroid Rank Function Theory **Field B** (complexity object): Karchmer-Wigderson Game Communication Complexity

Statement:

For a Boolean function f , the communication complexity of the Karchmer-Wigderson game is $\Theta(\text{rank}(f))$, where $\text{rank}(f)$ is the rank of the matroid constructed from f by taking clauses as elements and defining independence based on consistency (i.e., subsets of clauses whose conjunction is satisfiable).

Rationale (proposer's reasoning):

The rank of the matroid captures the minimal number of clauses needed to ensure consistency, which directly relates to the information-theoretic lower bounds on communication required to resolve the function's value in the Karchmer-Wigderson game. This conjecture bridges matroid theory's structural properties with communication complexity, avoiding barriers like algebrization by focusing on combinatorial rank rather than algebraic invariants.

Taxonomy category: Karchmer-Wigderson (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f41a7073beb11c2b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = set(random.sample(range(1, n + 1), random.randint(1, n)))
        cnf.append(clause)
    return cnf

def matroid_rank(cnf):
    independent_sets = [{frozenset()}]
    for clause in cnf:
        new_independent_sets = set()
        for s in independent_sets:
            if all(x not in s for x in clause):
                new_independent_sets.add(s | {frozenset(clause)})
        independent_sets.update(new_independent_sets)
    return max(len(s) for s in independent_sets)

def karchmer_wigderson_communication_complexity(cnf):
    n = len(cnf[0])
    rank_value = matroid_rank(cnf)
    # Simulate deterministic protocol (simplified version)
    communication_cost = 2 * rank_value
    return communication_cost

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, 2 * n)
    cnf = generate_cnf(n, m)

    matroid_rank_value = matroid_rank(cnf)
    comm_complexity = karchmer_wigderson_communication_complexity(cnf)

    return {
```

```

        "metric_name": "communication_complexity",
        "metric_value": comm_complexity,
        "instances_tested": 1,
        "conjecture_holds": abs(comm_complexity - matroid_rank_value) <= 1, # Allow small constant
        "counterexample": "" if conjecture_holds else f"Graph with n={n}, A={','.join(map(str, c
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or list(range(2, 30 * 100 + 1, 100))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0515a984.py", line 65, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0515a984.py", line 48, in run_trial
    matroid_rank_value = matroid_rank(cnf)
                          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0515a984.py", line 31, in matroid_rank
    new_independent_sets.add(s | {frozenset(clause)})
TypeError: unhashable type: 'set'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unhashable type error, preventing data collection. Support fraction cannot be evaluated without successful trials. | next: Fix the `matroid_rank` function to use `frozensets` instead of `sets` for hashable storage

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136380
2	propose	ollama_remote	qwen3:8b	0	0	196654
3	preregistration	ollama_remote	qwen3:8b	0	0	31677
4	novelty	ollama_remote	qwen3:8b	0	0	27683
5	novelty	ollama_remote	qwen3:8b	0	0	20397
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11107
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9603
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7626
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8255
10	judge	ollama_remote	qwen3:8b	0	0	107221

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 556603 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/028167524e77.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/028167524e77.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/028167524e77.tar.gz` (if generated)